Lab 1:Data Lab 实验报告

梁宇坤 24307110006

一. 题目实现逻辑

1.

```
int signMask(void) {
  return 1 << 31;
}
```

直接将 1 左移 31 位至最高位即可。

2.

```
int bitXor(int x, int y) {
  return ~(x & y) & ~(~x & ~y);
}
```

这样的表达下，只有当某位置 x 与 y，x 取反与 y 取反均不相同时才会输出 1，若该位置 x 与 y 数字同为 0 或 1 均输出 0。

3.

```
int negativePart(int x){
  return (x >> 31) & (~x + 1);
}
```

X 右移 31 得到全部为 0（对应正数）或 1（对应负数）的掩码，再与-x（取反加 1）作与运算。

4.

```
int copyByteWithin(int x, int src, int dst) {
  int srcByte = (x >> (src << 3)) & 0xFF;
  int cleared = x & ~(0xFF << (dst << 3));
  return cleared | (srcByte << (dst << 3));
}
```

将 src 对应的位数右移到尾部并和 0xFF 做与运算复制得到对应数字，接着通过 0xFF 左移取反与 x 的与运算完成对 dst 对应位置的清零，最后通过或运算把复制的数字粘贴过去。

5.

```
int logicalShift(int x, int n) {
  return (x >> n) & ~(((1 << 31) >> n) << 1);
}
```

原数做右移，构造一个高位为 0，低位全 1 的掩码，运算后去除多出的 1。

6.

```
int swapNibblePairs(int x) {
  int mask = 0x0F | (0x0F << 8) | (0x0F << 16) | (0x0F << 24);
  return ((x & mask) << 4) | ((x >> 4) & mask);
}
```

把每个字节的低 4 位和高 4 位分别用掩码抠出来，低 4 位左移 4 格、高 4 位右移 4 格，再拼回去。

7.

```
int secondLowestZeroBit(int x) {
  int y = x | (x + 1);
  return ~y & (y + 1);
}
```

把最低的 0 位填成 1，再找这个新数里最低的 0 位, 就是原数的第二低 0 位。

8.

```
int oddParity(int x) {
  x = x ^ (x >> 16);
  x = x ^ (x >> 8);
  x = x ^ (x >> 4);
  x = x ^ (x >> 2);
  x = x ^ (x >> 1);
  return !(x & 1);        你, 51秒钟前 • 未提交的更改
}
```

把 32 位不断对半异或折叠，最后 1 位就是 1 的个数奇偶性，再取反得到题目要求的偶数为 1,奇数为 0。

9.

```
int rotateRightBits(int x, int n) {
  int logical = (x >> n) & ~((1 << 31) >> n) << 1);
  int wrap = x << ((~n + 1) & 31);
  return logical | wrap;
}
```

逻辑右移 n 位再或上低 n 位左移 32-n 位绕回高位。用(-n) & 31 代替 32-n 可以解决边界问题。

10.

```
int roundEvenPow2(int x, int n) {
  int mask = (1 << n) + ~0;
  int half = 1 << (n + ~0);
  int low  = x & mask;
  int base = x >> n;

  int gt = ~((low + ~half) >> 31) & 1;
  int eq = !(low ^ half);
  int carry = gt | (eq & (base & 1));

  return (base + carry) << n;
}
```

把 x 拆成 base+ 低 n 位 low。low 超过中点（或正好在中点且 base 为奇数）时让商加 1，最后乘回。

11.

```
int midpointTowardFirst(int x, int y) {
  int mid = (x & y) + ((x ^ y) >> 1);
  int diff = x + ~y + 1;
  int sx = x >> 31;
  int sy = y >> 31;
  int sd = diff >> 31;
  int xLtY = (sx & ~sy) | (~(sx ^ sy) & sd);
  int xGtY = ~xLtY;
  int adjust = xGtY & (x ^ y) & 1;              你, 43秒钟前 • 未提交的更改
  return mid + adjust;
}
```

先无溢出算 floor 平均值；当 x+y 为奇数时，若 x>y 需要+1（取 ceil），否则取 mid。x > y 的判断用符号位和差值符号组合，避开溢出。

12.

```
int isBetweenEitherOrder(int x, int a, int b) {
  int sa = a >> 31, sb = b >> 31;
  int diff = a + ~b + 1;
  int sd = diff >> 31;      你, 1分钟前 • 未提交的更改
  int abSign = sa ^ sb;
  int aLtB = (abSign & sa) | (~abSign & sd);

  int lo = (a & aLtB) | (b & ~aLtB);
  int hi = (b & aLtB) | (a & ~aLtB);

  int slo = lo >> 31, sx = x >> 31;
  int d1 = x + ~lo + 1;
  int sd1 = d1 >> 31;
  int loSign = sx ^ slo;
  int geLo = ((loSign & ~sx) | (~loSign & ~sd1)) & 1;

  int shi = hi >> 31;
  int d2 = hi + ~x + 1;
  int sd2 = d2 >> 31;
  int hiSign = shi ^ sx;
  int leHi = ((hiSign & ~shi) | (~hiSign & ~sd2)) & 1;

  return geLo & leHi;
}
```

用掩码选 min,max，再检查 x >= min 和 x <= max，两个都成立就返回 1。

13.

```
int mul5Sat(int x) {
  int sx = x >> 31;
  int prod = (x << 2) + x;
  int upper = (0x19 << 8) | 0x99;
  upper = (upper << 8) | 0x99;
  upper = (upper << 8) | 0x99;
  int gtUpper = ~sx & ~((x + ~upper) >> 31);
  int ltLower = sx & ((x + upper) >> 31);
  int mask = gtUpper | ltLower;
  int satVal = (1 << 31) ^ ~sx;
  return (prod & ~mask) | (satVal & mask);
}      Lik0601, 2小时前 • Initial commit
```

x*5 不溢出的范围是 [-429496729, 429496729]；超出则按 x 符号返回 INT_MAX 或 INT_MIN。判断越界用符号位 + 差值符号位的位运算完成。

14.

```
int classifyAdd3(int x, int y, int z) {
  int q = (x >> 2) + (y >> 2) + (z >> 2);        你, 54秒钟前 • 未提交的更改
  int r = (x & 3) + (y & 3) + (z & 3);
  int h = q + (r >> 2);
  return (h + (1 << 29)) >> 30;
}
```

把三个数分别拆成除以 4 的商和余数，算出总和除以 4 的向下取整值，再加上 2 的 29 次方并算术右移 30 位，就得到溢出分类的 −1、0 或 1。

15.

```
unsigned floatScaleThreeHalves(unsigned uf) {
  unsigned s = uf & 0x80000000;
  unsigned e = (uf >> 23) & 255;
  unsigned m = uf & 0x7fffff;
  if (e == 255)
      return uf;
  if (e == 0) {
      m = m + (m << 1);
      return s | ((m >> 1) + ((m & 3) == 3));
    }
  m = (m | 0x800000) * 3;
  if (m < 0x2000000) {
      m = (m >> 1) + ((m & 3) == 3);
    } else {
      m = (m >> 2) + (((m & 3) > 2) || ((m & 7) == 6));
      e = e + 1;
    }
  if (m == 0x1000000) {
      m = m >> 1;
      e = e + 1;
    }
  if (e >= 255)
      return s | 0x7f800000;
  return s | (e << 23) | (m & 0x7fffff);      Lik0601, 2小时前 • Initial commit
}
```

拆出符号、指数和尾数，将有效尾数乘 3 后右移 1 或 2 位并调整指数，按“就近舍入、恰好一半取偶数”处理丢弃位，最后处理进位溢出及 NaN、无穷大、正负零等特殊情况。

16.

```
unsigned floatRoundEven(unsigned uf) {
    unsigned s = uf & 0x80000000;
    unsigned a = uf & 0x7fffffff;
    unsigned e = (uf >> 23) & 255;
    unsigned unit, mask, rem, half, result;
    if (e >= 150)
        return uf;
    if (a <= 0x3f000000)
        return s;
    if (a < 0x3f800000)
        return s | 0x3f800000;
    unit = 1u << (150 - e);
    mask = unit - 1;
    rem = uf & mask;
    half = unit >> 1;
    result = uf & ~mask;
    if (rem > half || (rem == half && (result & unit)))
        result = result + unit;      你, 42秒钟前 • 未提交的更改
    return result;
}
```

根据指数找到整数与小数的分界，清除小数位；若小数大于 0.5，或等于 0.5 且整数部分为奇数，就将绝对值加 1，最后保留原符号，并单独处理零、NaN 和无穷大。

17.

```c
unsigned float_i2f(int x) {
  unsigned u = x;
  unsigned s = u & 0x80000000;
  unsigned e = 158;   /* 127 + 31 */
  unsigned m, rem, result;
  if (!x)
      return 0;
  if (s)
      u = ~u + 1;
  while (!(u & 0x80000000)) {
      u = u << 1;
      e = e - 1;
  }
  m = (u >> 8) & 0x7fffff;
  rem = u & 255;
  result = s | (e << 23) | m;
  if (rem > 128 || (rem == 128 && (m & 1)))
      result = result + 1;
  return result;      Lik0601, 2小时前 • Initial commit
}
```

先提取符号并取绝对值，根据最高位的 1 确定指数，保留前 24 位有效数字，对丢弃部分按“就近舍入、半数取偶”处理，最后拼接符号、指数和尾数。

18.

```c
int bitCount(int x) {
  int m1 = 0x55 | (0x55 << 8);
  int m2 = 0x33 | (0x33 << 8);
  int m4 = 0x0f | (0x0f << 8);
  m1 = m1 | (m1 << 16);
  m2 = m2 | (m2 << 16);
  m4 = m4 | (m4 << 16);
  x = (x & m1) + ((x >> 1) & m1);
  x = (x & m2) + ((x >> 2) & m2);
  x = (x + (x >> 4)) & m4;
  x = x + (x >> 8);
  x = x + (x >> 16);
  return x & 0x3f;
}
```

用掩码和移位分组统计：先算每 2 位中 1 的个数，再逐级合并成 4 位、8 位、16 位、32 位的计数，最后取低 6 位得到总数。

19.

```c
int bitReverse(int x)
{
int m = (0xff << 8) | 0xff;                  你, 1分钟前 • 未提交的更改
x = ((x >> 16) & m) | (x << 16);
m = m ^ (m << 8);
x = ((x >> 8) & m) | ((x & m) << 8);
m = m ^ (m << 4);
x = ((x >> 4) & m) | ((x & m) << 4);
m = m ^ (m << 2);
x = ((x >> 2) & m) | ((x & m) << 2);
m = m ^ (m << 1);
x = ((x >> 1) & m) | ((x & m) << 1);
return x;
}
```

先交换高低 16 位，再依次交换相邻的 8、4、2、1 位分组；掩码从小常数构造，并通过“自身异或左移值”逐级生成，恰好用 34 次操作完成反转。

二．实验结果截图

```
root@ics:~/ics-datalab-1# ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 11/18 operators
secondLowestZeroBit: 5/8 operators
oddParity: 12/56 operators
rotateRightBits: 11/16 operators
roundEvenPow2: 20/24 operators
midpointTowardFirst: 21/32 operators
isBetweenEitherOrder: 47/48 operators
mul5Sat: 13/30 operators
classifyAdd3: 15/52 operators
floatScaleThreeHalves: 37/60 operators
floatRoundEven: 21/65 operators
float_i2f: 20/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

```
.root@ics:~/ics-datalab-1# ./btest
Score   Rating  Errors  Function
 1      1       0       signMask
 2      2       0       bitXor
 3      3       0       negativePart
 4      4       0       copyByteWithin
 4      4       0       logicalShift
 4      4       0       swapNibblePairs
 4      4       0       secondLowestZeroBit
 5      5       0       oddParity
 5      5       0       rotateRightBits
 5      5       0       roundEvenPow2
 5      5       0       midpointTowardFirst
 7      7       0       isBetweenEitherOrder
 7      7       0       mul5Sat
 7      7       0       classifyAdd3
 7      7       0       floatScaleThreeHalves
 10     10      0       floatRoundEven
 10     10      0       float_i2f
 10     10      0       bitCount
 10     10      0       bitReverse
Total points: 110/110
```